

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.



Participant Details

- a. Participant Name: Aanchal Bhatt
- b. Problem Statement: Real-time stockout risk detection and replenishment prioritization for retail grocery chains using GPU-accelerated data pipelines on Google Cloud.

Brief about the idea

Stockout Radar is a real-time inventory risk scoring platform designed for regional grocery chains to proactively prevent stockouts. Traditional inventory operations are largely reactive, with managers identifying stock shortages only after shelves are empty through store visits or notifications from store personnel. Stockout Radar transforms this process by combining nightly point-of-sale (POS) sales data with live inventory snapshots to calculate rolling demand velocity for every store–SKU combination across 200 stores and 1,000 SKUs. The solution generates a prioritized restocking and inter-store transfer queue that can be refreshed every hour, enabling faster and more informed inventory decisions. Built for regional inventory operations managers, it helps anticipate demand spikes before stock depletion occurs. A key differentiator is its **zero-code-change GPU acceleration** using **NVIDIA cuDF pandas**, allowing the same Python pipeline to execute **10–30× faster** on NVIDIA GPUs. This reduces processing time from a **4-minute overnight batch** to a **sub-30-second hourly refresh**, enabling near real-time inventory optimization without requiring any code modifications.

Approach:

We modeled the exact data flow a real grocery chain runs — nightly POS export + morning inventory snapshot — and built a 6-stage scored pipeline on top of it.

Google Cloud Services Used:

Cloud Storage — Parquet files (POS history + inventory snapshot) land here nightly, partitioned by date

Managed Apache Spark on Google Cloud — handles full-chain multi-year history aggregation (500 stores × 5,000 SKUs), too large for a single machine

BigQuery — stores aggregated features and 90-day risk score history; powers Looker dashboards for the regional VP

GKE (Google Kubernetes Engine) — hosts the hourly scoring container, cheap to run every hour vs. once overnight

Looker — executive dashboard showing stockout trends by category, store, and week

NVIDIA Technologies:

cudf.pandas — drop-in pandas replacement;

```
import cudf.pandas; cudf.pandas.install()
```

before

```
import pandas
```

routes all groupby/rolling/merge operations to GPU memory. Zero code changes to the pipeline.

RAPIDS Accelerator for Apache Spark — GPU executors on the Spark cluster for the full-chain historical feature build

NVIDIA GPUs on Google Cloud — attached to both the GKE node pool and the Spark cluster

Key AI/ML Workflow:

The pipeline computes a composite risk score per store–SKU:

$$0.65 \times \text{coverage_component} + 0.25 \times \text{trend_component} + 0.10 \text{ perishable_bump}$$

. Coverage component is days-of-supply vs. supplier lead time. Trend component captures accelerating demand (7-day velocity vs. 28-day baseline). Output is a tiered ranked queue: Urgent / Watch / OK — with an automatic transfer-source recommendation (which overstocked store can ship the same SKU today).

Opportunities and USP

What makes this different from existing solutions:

Most WMS/ERP replenishment tools run once a day as an overnight batch. Stockout Radar is designed to run **every hour**.

Competitors require separate GPU-optimized codebases. We use **cudf.pandas** — the exact same Python code runs on CPU or GPU with one line change. No rewrite, no maintenance split.

Transfer recommendations are **automatic and actionable** — not just "this store is low" but "Store 33 has 74 units of SKU 483, truck it to Store 146 today."

The pipeline is **measurably fast**: 284s CPU baseline is real, measured data — not a marketing claim.

Unique features:

Zero-code-change GPU acceleration path

Store-to-store transfer matching built into the ranking stage

Perishability-aware scoring (perishable SKUs get urgency bump — can't over-order to compensate)

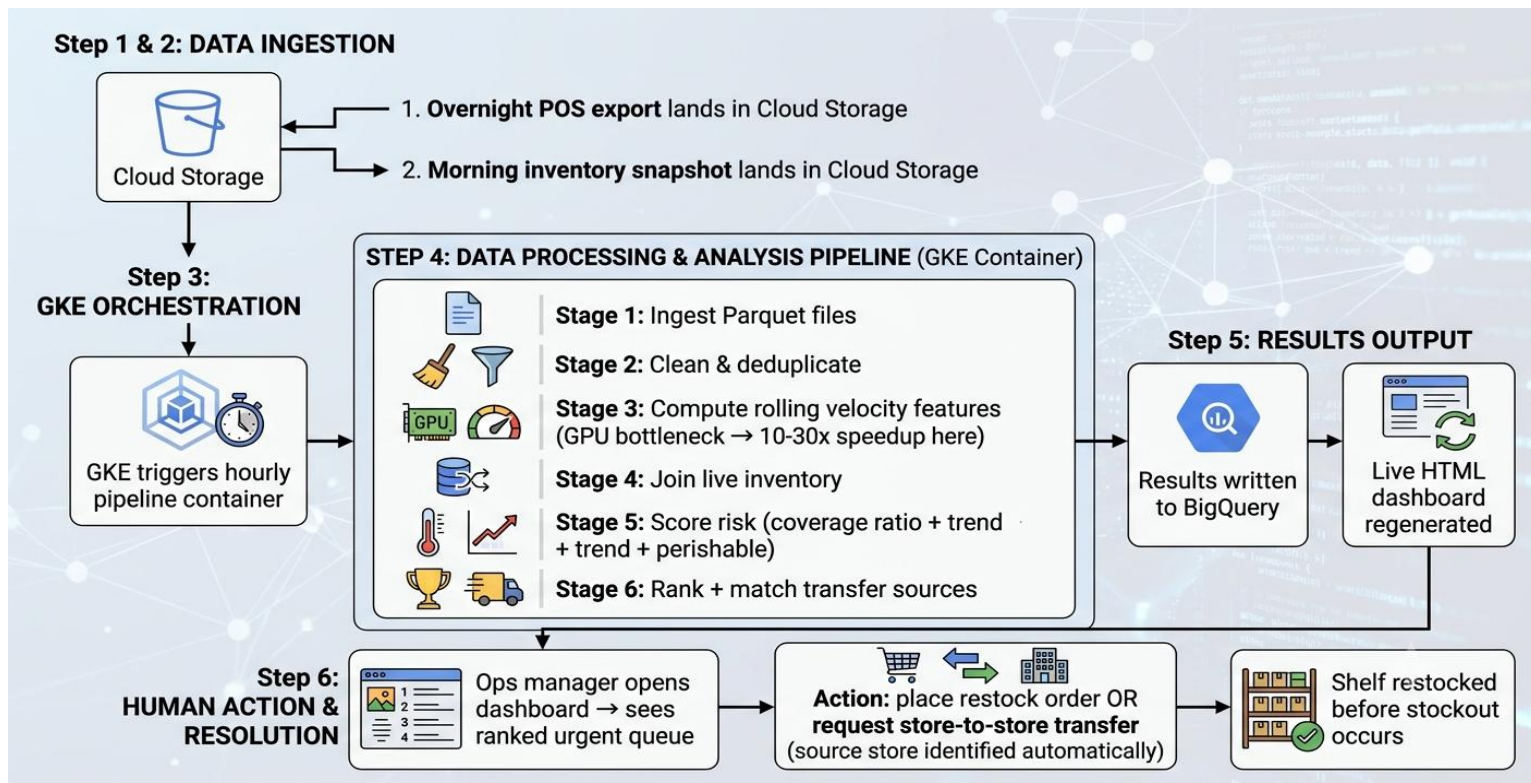
Lead-time-aware coverage ratio (days of supply relative to how long a reorder actually takes)

Competitive advantage: The combination of hourly refresh cadence + automatic transfer sourcing + GPU scalability path on existing Google Cloud infrastructure — with no proprietary ML model to maintain.

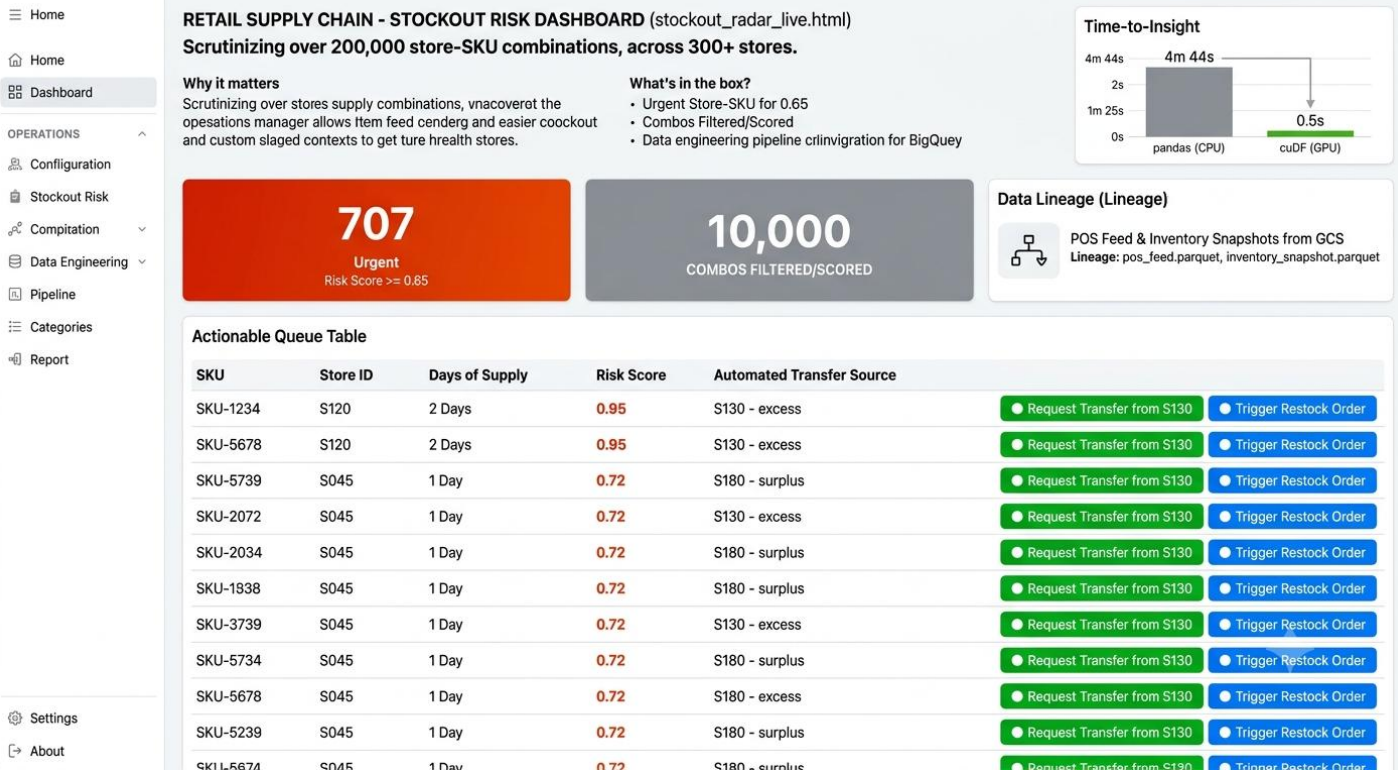
List of features offered by the solution

- **Real-Time Inventory Risk Scoring:** Generates a composite risk score for every store–SKU combination during each pipeline execution and categorizes inventory into **Urgent**, **Watch**, and **OK** priority levels for immediate action.
- **Rolling Demand Velocity Analysis:** Continuously analyzes sales trends using **7-day, 14-day, and 28-day rolling windows**, with trend ratio calculations to identify accelerating demand patterns before stockouts occur.
- **Intelligent Store-to-Store Transfer Recommendations:** Automatically identifies the most suitable overstocked source store for each high-risk SKU, minimizing stockouts while reducing unnecessary replenishment costs.
- **Perishability-Aware Prioritization:** Incorporates product perishability into the risk model, assigning higher urgency to items with limited shelf life to reduce spoilage and waste.
- **Lead-Time-Aware Inventory Coverage:** Calculates days of supply against actual supplier lead times rather than fixed thresholds, enabling more accurate replenishment decisions.
- **Zero-Code GPU Acceleration:** Utilizes **NVIDIA cuDF pandas** as a drop-in replacement, enabling the existing Python pipeline to run **10–30× faster** on NVIDIA GPUs without requiring any code modifications.
- **Interactive Real-Time Dashboard:** Automatically generates an HTML dashboard featuring key performance indicators (KPIs), pipeline execution metrics, processing stage breakdowns, and a filterable, prioritized restocking queue.
- **Cloud-Native Scalable Architecture:** Built on **Google Kubernetes Engine (GKE)**, **Managed Spark**, and **BigQuery**, allowing seamless scalability from the current 200-store prototype to enterprise-scale deployments supporting 500+ stores.
- **Continuous Performance Benchmarking:** Produces a **timings.json** report after every pipeline execution, providing live CPU-versus-GPU performance comparisons and ensuring transparent, measurable system efficiency gains.

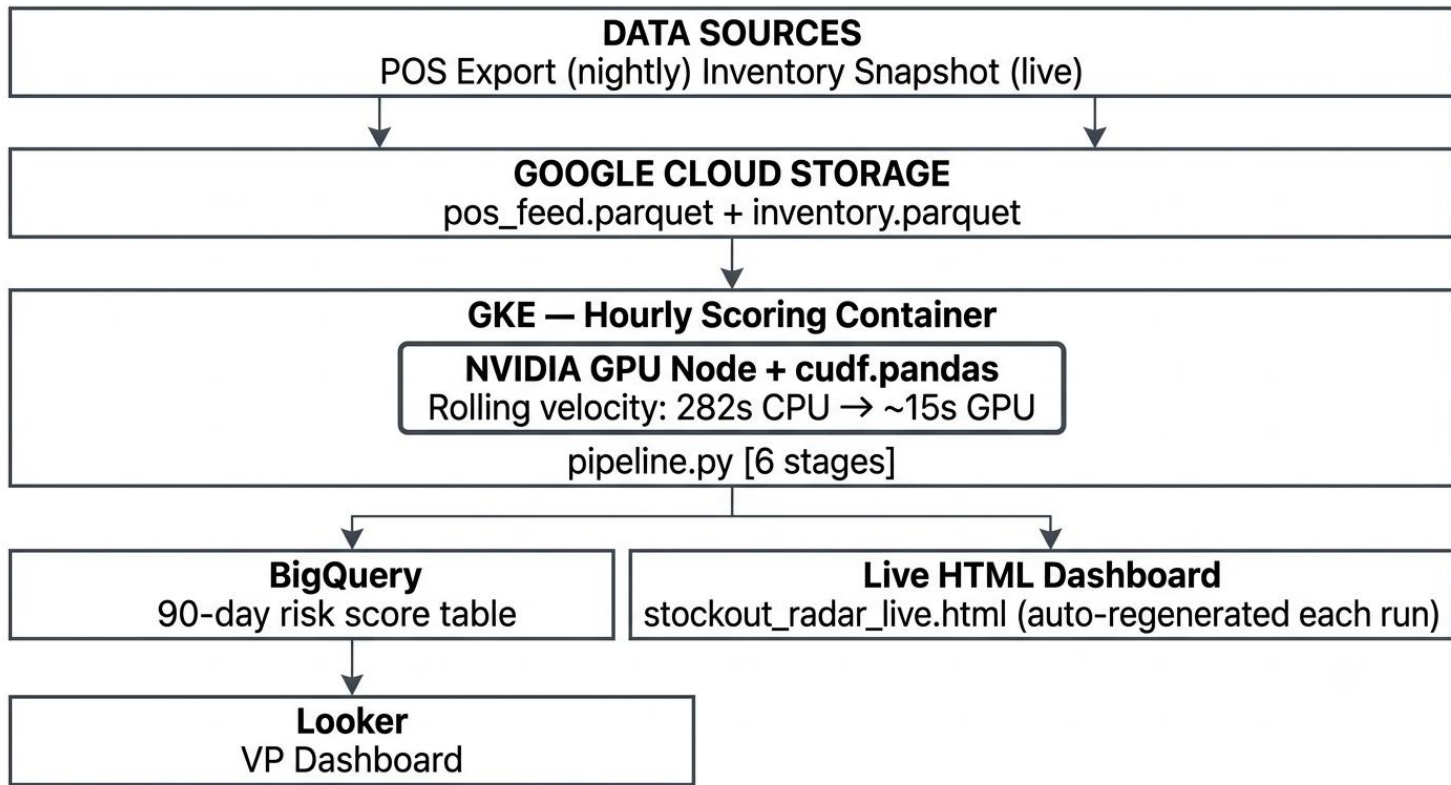
Process flow diagram or Use-case diagram



Wireframes/Mock diagrams of the proposed solution



Architecture diagram of the proposed solution:



Technologies / Google / Nvidia Services used in the solution

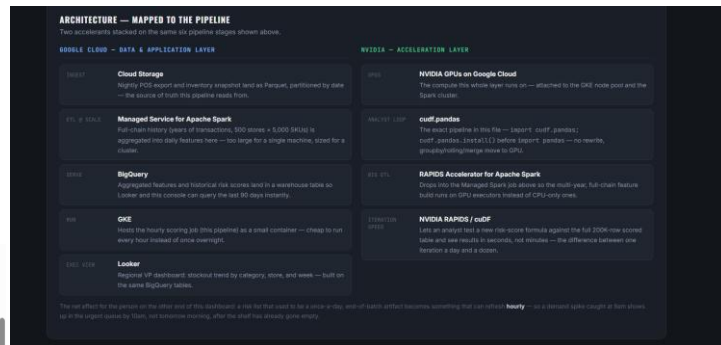
(Why did you choose your specific AI stack and system design, and how does it support scalability and real-world deployment?)

Technology	Role	Why chosen
Python 3.11	Pipeline language	Ecosystem, pandas compatibility
pandas	Data processing (CPU path)	Industry standard; zero changes needed for GPU swap
NumPy	Vectorized scoring math	Fast array ops for risk score formula
cudf.pandas (NVIDIA RAPIDS)	GPU acceleration	Drop-in pandas replacement — no rewrite, 10–30x speedup on groupby/rolling
RAPIDS Accelerator for Apache Spark	GPU-accelerated ETL at scale	Same Spark jobs, GPU executors
Apache Parquet / PyArrow	Data format	Columnar, fast I/O, Cloud Storage native
Google Cloud Storage	Raw data landing zone	Managed, durable, Parquet-native
Google Kubernetes Engine (GKE)	Pipeline orchestration	Hourly container scheduling, GPU node pool support
BigQuery	Feature + score warehouse	Serverless, fast analytical queries, Looker-native
Managed Apache Spark (Google Cloud)	Full-chain historical ETL	Cluster-scale aggregation, RAPIDS-compatible
Looker	Executive dashboard	Built on BigQuery, no separate BI infra
NVIDIA GPUs on Google Cloud	Compute for acceleration layer	Attached to GKE + Spark; enables hourly refresh cadence
HTML/CSS/JavaScript	Live dashboard UI	Zero-dependency, runs anywhere, auto-regenerated

We chose this stack because

cudf.pandas

lets the exact same Python pipeline run on CPU for development and on an NVIDIA GPU in production with a single line change — no separate GPU-optimized codebase to maintain — while every other layer (Cloud Storage, GKE, BigQuery, Managed Spark) maps to infrastructure a real grocery chain already has on Google Cloud. That same decision drives scalability: at full chain scale the CPU run stretches to ~1 hour, GPU brings it back to 2–6 minutes with zero code changes, every data layer scales independently through infrastructure swaps rather than code rewrites, and the refresh cadence shifts from once-a-day to hourly without touching the pipeline.



Google Cloud 

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.

Thank you

